Traduce Señas:

Installation Manual

Juan Mauricio Suaza Solórzano

Luis Alejandro Duarte Aldana

Juan Camilo Fierro Esquivel

Instructor: Karol Daniela Correa

October 7, 2026

Table of Contents

Introduction 3

1. Key Concepts 4

    1.1 General Characteristics (Ports) 4

    1.2 Database Platform (Migrations) 4

    1.3 Configuration Variables (Names Only) 5

2. Installation 6

    2.1 Requirements 6

    2.2 Getting the Code 6

    2.3 Installation With Docker (Recommended Route) 7

    2.4 Alternative Route Without the Uncommitted Changes 9

    2.5 Development Mode Without Docker 9

        2.5.1 Port 3000 Conflict 10

    2.6 Installation on a Production Server 10

3. Configuration and First Use 10

    3.1 Checking That It Works 10

    3.2 Creating and Activating the First Account 12

    3.3 First Administrator 12

    3.4 Resetting the Database (Deletes All Data) 12

4. Operation 13

5. Common Problems 13

References 15

Introduction

This manual explains how to get Traduce Señas running **on a local computer** with Docker. It does not cover production. It does not include configuration values: you define the keys and passwords yourself, and they are **never** written in this manual or in the repository.

Important warning

Today the **complete Docker installation** depends on changes that **are not in any commit yet**:

- The new `Dockerfile` with `HEALTHCHECK` and the new `.dockerignore` exist only as **uncommitted** changes on the Backend disk (local branch `chore/docker-backend`). That branch is **older** than `develop`: it has no email verification, no `/api/iam`, no `users.status` and none of migrations 003-006. **It is not useful for running the system described in these manuals.** The committed Dockerfile is identical in both branches, which is why the uncommitted changes can be carried over to `develop` (section 2.3).
- The `Dockerfile` that is in `origin/develop` **does not define `HEALTHCHECK`**, and `docker-compose.yml` makes the frontend wait for the backend to be "healthy" (`service_healthy`). With the `develop` Dockerfile, the frontend **does not start** with `docker compose up`.
- The `docker-compose.yml`, `build-frontend.sh` and `build-backend.sh` files **are not in any of the three repositories**: they live in the local `infra/` folder of the project folder (`sena/`). Anyone who only clones the repos does not have them; ask the team for them.

Section 2.3 is the recommended route and section 2.4 an alternative.

1. Key Concepts

1.1 General Characteristics (Ports)

Table 1

General Characteristics (Ports)

Component	On your computer	Inside Docker
Web (nginx)	`http://127.0.0.1:8080`	port 80
Backend (API)	`http://127.0.0.1:3000`	port 3000
PostgreSQL 16	`127.0.0.1:5433`	port 5432

All ports are published **only on `127.0.0.1`**: nobody else on the network can connect. To test the app from a real phone you would have to remove `127.0.0.1:` from the backend port, and do so only on trusted networks.

The web (nginx; F5, Inc., n.d.) serves the application and forwards everything starting with `/api/` to the backend. There is no Redis or MongoDB.

1.2 Database Platform (Migrations)

The compose file mounts `../repos/Backend/migrations` (relative to `infra/`) at `/docker-entrypoint-initdb.d`. PostgreSQL runs those `.sql` files in order **only when the volume is empty** (the first time; The PostgreSQL Global Development Group, n.d.).

- In `origin/develop` there are seven migrations: `000_users.sql`, `001_translations.sql`, `002_sign_templates.sql`, `003_iam_roles_permissions.sql`, `004_analytics_usage_events.sql`, `005_auth_email_normalization_user_tokens.sql` and `006_users_status_email_verification.sql`.
- **The schema depends on the branch that is on disk** in `repos/Backend/` (which must be an up-to-date `develop`, `2c4e62d`): otherwise migrations 003 to 006 are missing and the roles, statistics and email verification features will not work. The local `chore/docker-backend` branch (and the outdated local `develop`, `ad9815d`) only have migrations 000 to

002: with them sign-up fails, `/api/iam/me/access` returns 404 and the first-use SQL statements (`users.status`, `user_roles`) do not work.

- If you change branches or a migration fails, you must **reset the volume**.

1.3 Configuration Variables (Names Only)

The backend reads these variables (`Backend@develop:src/shared/config/config.ts`). They are defined in `repos/Backend/.env`, which is **never** pushed to git:

Table 2

Configuration Variables (Names Only)

Variable	Purpose
`PORT`	Backend port (3000 if missing).
`DB_HOST`, `DB_PORT`, `DB_USER`, `DB_PASSWORD`, `DB_NAME`	PostgreSQL connection. **With Docker Compose, the compose file sets them for you**; you do not need to put them in `.env`.
`JWT_SECRET`	Secret used to sign sessions. **Required.** Choose a long, random text.
`EMAIL_USER`, `EMAIL_PASS`	Email account (`gmail` service) used to send the verification codes.

Steps:

1. Copy the example file: `cp repos/Backend/.env.example repos/Backend/.env` (from `sena/`).
2. Open `repos/Backend/.env` and fill in `JWT_SECRET` with your own value.
3. If you want the code emails to be delivered, fill in `EMAIL_USER` and `EMAIL_PASS` (Gmail usually requires an "app password"). If you don't, sign-up works but the code does not arrive: activate the account manually (section 3.2).

The database credentials in the compose file are development values defined in `docker-compose.yml` itself. Do not copy them into any document or use them on a real server.

2. Installation

2.1 Requirements

Table 3

Requirements

You need	Details
Git	To clone the repositories.
Node.js 20 or later	JavaScript runtime (OpenJS Foundation, n.d.). Only for development mode without Docker (section 2.5). The images use Node inside the container.
Docker Desktop	With `docker` and `docker compose` available (Docker Inc., n.d.).
Disk space	Reserve several free GB: Docker images, `node_modules` and the recognition model.
Internet	To download images and dependencies, and for the browser to fetch MediaPipe when using the camera.
A bash-like terminal	On Windows, Git Bash. The `build-*.sh` scripts are bash.
An email account to send codes	Optional for testing (see section 1.3). Sending uses the `gmail` service of `nodemailer`.

2.2 Getting the Code

This is the **recommended** layout (the one used on the development team's machine): a `sena/` folder with the three repositories inside `repos/` and the infrastructure files in `infra/`. `sena/` is not a git repository. Your teammates may have a different layout: the compose file and the scripts are outside the repos and only need to find `Backend` and `Frontend` through relative paths (the compose file uses `../repos/...` from `infra/`; if you change the layout, adjust those paths).

```
sena/
  repos/
    Backend/    <- github.com/mauro-1508/backendTS      (develop branch)
    Frontend/   <- github.com/tadeo77789/frontendTS     (develop branch)
    Docs/       <- github.com/code-sena/trans-sl-docs   (main branch)
  infra/
    docker-compose.yml    <- NOT in any repo; ask the team for it
    build-frontend.sh     <- same
    build-backend.sh      <- same
```

```
mkdir -p sena/repos sena/infra && cd sena
git clone -b develop https://github.com/mauro-1508/backendTS.git
repos/Backend
git clone -b develop https://github.com/tadeo77789/frontendTS.git
repos/Frontend
git clone -b main https://github.com/code-sena/trans-sl-docs.git
repos/Docs
```

The commands in this manual are run **from the `sena/` root** (the git and `cp` ones) or **from `sena/infra/`** (the `docker compose` ones and the `build-*.sh` scripts). Each block states where to run it from.

In the Backend repository, the `main` branch only has a README. The code is in `develop`: check with `git -C repos/Backend branch --show-current`.

Cloud-synced folders (OneDrive and similar). If you put `sena/` in a synced folder, Docker (BuildKit) cannot read some of its files. That is why the `build-*.sh` scripts exist (section 2.3). Also, mark `repos/Backend/migrations` as "Always keep on this device"; otherwise the migrations mount may appear empty.

2.3 Installation With Docker (Recommended Route)

Requirements: `docker-compose.yml` and `build-*.sh` in `infra/`, and the Backend **on an up-to-date `develop` branch** (commit `2c4e62d`), keeping the uncommitted changes to `Dockerfile` and `.dockerignore`.

Step A. Switch the Backend to `develop` and update it. The local `develop` may be behind `origin/develop`:

(from `sena/`)

```
git -C repos/Backend checkout develop
git -C repos/Backend pull --ff-only origin develop
git -C repos/Backend rev-parse --short HEAD     # must show 2c4e62d
```

The uncommitted changes to `Dockerfile` and `.dockerignore` are kept because the committed version of those files is the same in `chore/docker-backend` and in `develop`. If Git refuses because of other modified files (for example `.gitignore`, which does differ between the branches), save a copy of that file, discard your changes to it and repeat the command. After the

`pull`, add back to the end of `.gitignore` the lines from your copy that are missing (the `liquibase.properties` ones): they protect files with credentials and must not be lost.

Step B. Build and start, from `sena/infra/`:

```
cd infra
./build-frontend.sh
./build-backend.sh --dockerfile-from develop
docker compose up -d
```

What each command does:

1. `./build-frontend.sh` builds the `traduce-senas-frontend:latest` image. It sends the files of `repos/Frontend/` to Docker as a `tar` (without `node_modules`, `.git`, `dist` or `.expo`) and builds the web app with Expo (Expo, n.d.) inside the image. It takes several minutes the first time.
2. `./build-backend.sh --dockerfile-from develop` builds `traduce-senas-backend:latest`. With that option, the current branch matches the "Dockerfile branch" and the script packages the working folder directly, that is, **with the uncommitted Dockerfile that has `HEALTHCHECK`**. Without the option, the script looks for the Dockerfile in `chore/docker-backend` and, being on `develop`, aborts because that committed Dockerfile is the same as `develop`'s.
    - The script **will always show** "AVISO: se esperaba feat/email-verification…": that branch no longer exists and the warning is harmless.
    - It also warns if there are uncommitted changes (they are included in the image, which is expected) and if `repos/Backend/.env` is missing.
3. `docker compose up -d` starts PostgreSQL, the backend and the web. The frontend waits for the backend to be healthy.

Why there are scripts instead of `docker compose build`: the project folder is usually cloud-synced and BuildKit cannot read those files (reparse points). The scripts avoid the problem by sending the context as a `tar`.

If you change the code, run the corresponding script again and then `docker compose up -d`.

2.4 Alternative Route Without the Uncommitted Changes

If you only have the three repos cloned (Backend on `origin/develop`) and the `docker-compose.yml`, follow these steps:

1. Build the images with the names the compose file uses:

   ```
   docker build -t traduce-senas-backend:latest repos/Backend
   docker build -t traduce-senas-frontend:latest repos/Frontend
   ```

   (These two commands are run from `sena/`. If `sena/` is in a cloud-synced folder, this step may fail for the reason explained above; move the project to a local folder, e.g. `C:\dev\sena`. The following `docker compose` commands are run from `sena/infra/`.)

2. Start the database and the backend first: `docker compose up -d postgres backend`.
3. Start the web without waiting for the "healthcheck" that the `origin/develop` Dockerfile does not define: `docker compose up -d --no-deps frontend`.

   On this route the backend **will not have `healthy` status** (there is no `HEALTHCHECK`): in 3.1, only check that it is `running` and that the API responds.

2.5 Development Mode Without Docker

Use it to program with automatic reload. You need Node 20+ and a PostgreSQL database.

1. **Database:** you can use the compose one: `docker compose up -d postgres` (from `infra/`; it runs at `127.0.0.1:5433`).
2. **Backend:** in `repos/Backend/`:

   ```
   npm install
   cp .env.example .env
   npm run dev
   ```

   In `.env` set `DB_HOST` (127.0.0.1), `DB_PORT` (5433), `DB_NAME` (traduce_senas), `DB_USER`, `DB_PASSWORD` (the compose ones), `JWT_SECRET`, and optionally `EMAIL_USER`/`EMAIL_PASS`. Other commands: `npm run build`, `npm start`, `npm test` (uses vitest).

3. **Frontend:** in `repos/Frontend/`: `npm install` and then `npm run web` (Expo). The terminal shows you the address. In development the web calls `http://localhost:3000/api`. Other commands: `npm start`, `npm run android`, `npm run ios`, `npm run typecheck`.

2.5.1 Port 3000 Conflict

The development backend uses port **3000**, the same one the `backend` container publishes. **If both run at the same time, they conflict.** Stop the container before `npm run dev`:

```
docker compose stop backend
```

(The Docker web uses 8080, so with the development backend on 3000 you can keep the Docker database.)

2.6 Installation on a Production Server

This version of the project does not include a procedure for installing on a production server: the system runs on a local computer, as described in the previous sections.

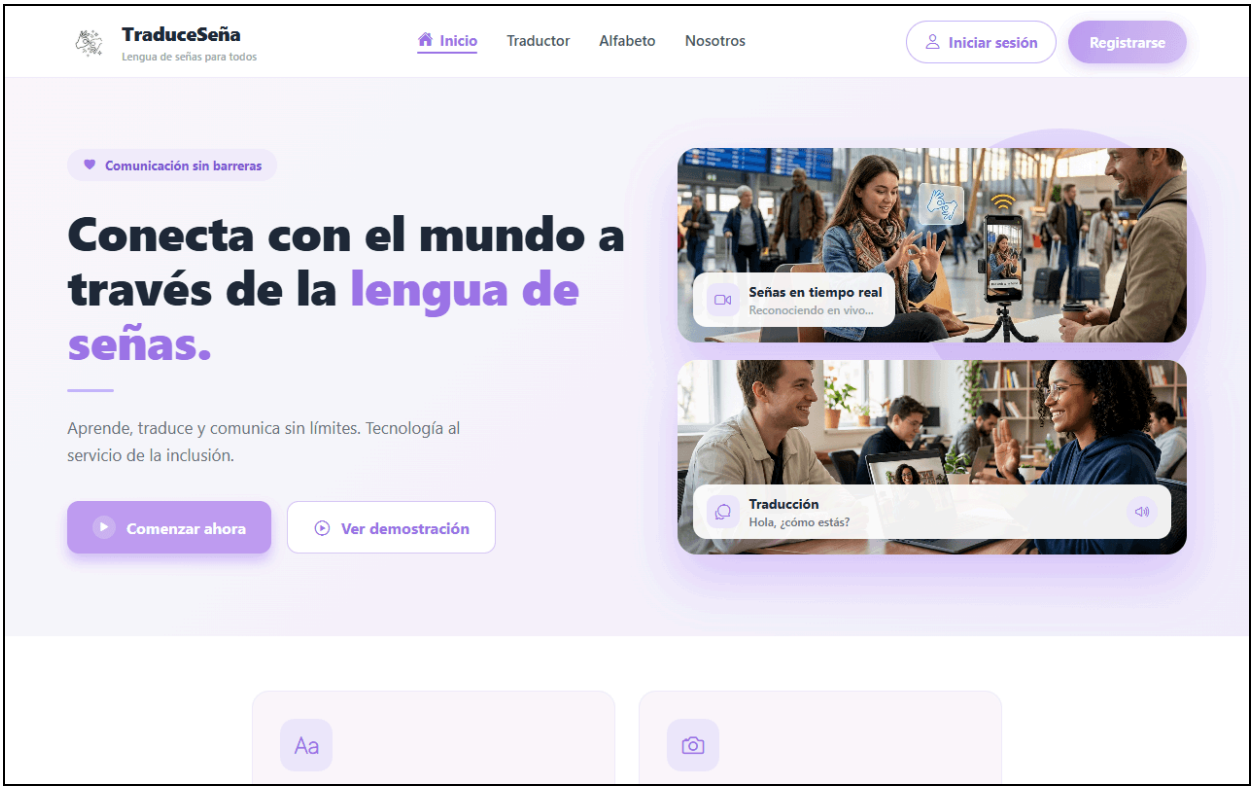
3. Configuration and First Use

3.1 Checking That It Works

1. Service status: `docker compose ps`. All three must be up (`running`). With the route in section 2.3, the backend and the database also show as `healthy`; with the route in section 2.4 the backend does not.
2. Open `http://127.0.0.1:8080`: the home page must load.

Figure 1

Application Home Page



3. Check that the API responds through the web: `curl http://127.0.0.1:8080/api/sign-templates` must return a JSON response (the list may be empty).
4. If something fails: `docker compose logs backend` and `docker compose logs postgres`.

3.2 Creating and Activating the First Account

1. On the web, tap **Sign up** and complete the **Create your Account** form.
2. New accounts stay **inactive** until the email is confirmed with a code. If you configured `EMAIL_USER` and `EMAIL_PASS`, the code arrives by email. Otherwise, activate the account manually:

```
docker compose exec postgres psql -U <database_user> -d
traduce_senas -c "UPDATE users SET status='ACTIVE' WHERE
email='<email>'"
```

(`<database_user>` is the PostgreSQL user defined in the compose file.)

3.3 First Administrator

It is done with an SQL statement, only once. It is explained step by step in the Administrator manual, section 3. In short, the `ADMIN` role is inserted for that email in the `user_roles` table.

3.4 Resetting the Database (Deletes All Data)

This deletes ALL the data in the database (accounts, history, templates). Make a backup first if you care about it: `docker compose exec postgres pg_dump -U <database_user> traduce_senas > backup.sql`

```
docker compose down -v
docker compose up -d
```

4. Operation

Table 4

Operation

I want to...	Command
Stop everything without deleting data	`docker compose stop`
Start again	`docker compose up -d`
Stop and remove containers (data is kept)	`docker compose down`
See backend messages	`docker compose logs -f backend`
Also delete the data	`docker compose down -v` (**deletes the data**)
Update the code	`git pull` in each repo (`repos/Backend`, `repos/Frontend`, `repos/Docs`), rebuild the images (section 2.3) and `docker compose up -d`. New migrations are not applied automatically to an existing database.

5. Common Problems

Table 5

Common Problems

Symptom	Likely cause	What to do
`./build-backend.sh` says the branch does not exist or that it "does not have the new Dockerfile committed"	You ran the script without `--dockerfile-from develop`: the `chore/docker-backend` branch does not exist or its Dockerfile is not committed	Run `./build-backend.sh --dockerfile-from develop` (section 2.3) or use section 2.4
`docker compose up` fails with the frontend "has no healthcheck"	The backend Dockerfile is the `develop` one, without `HEALTHCHECK`	Section 2.4, step 3

The frontend does not start (nginx: `host not found in upstream "backend"`)	Nginx needs the `backend` service to exist already	Start `backend` first; check `docker compose logs frontend`
The build fails with unreadable-file errors	Cloud-synced folder	Use the `build-*.sh` scripts or move the project to a local folder
Tables do not exist or roles/statistics are missing	Volume created with another branch, or a failed migration	Sections 1.2 and 3.4: reset the volume with the correct branch
Database error after the database name changed	The old volume has another database	`docker compose down -v` (deletes data)
The verification code does not arrive	`EMAIL_USER`/`EMAIL_PASS` not configured or rejected	Activate the account manually (3.2) or configure email
All users lose their session after a restart	`JWT_SECRET` changed or is empty	Set a fixed `JWT_SECRET` in `repos/Backend/.env`
Port 8080, 3000 or 5433 is in use	Another program is using it	Close that program or change the port in the compose file
The camera does not work on the web	The browser does not have permission	Accept the permission. Some browsers only allow the camera on `localhost`/`127.0.0.1` or over HTTPS
`npm run dev` fails with "EADDRINUSE"	Port 3000 is taken by the container	`docker compose stop backend`

References

Docker Inc. (n.d.). *Docker Compose*. Docker Docs. Retrieved October 7, 2026, from https://docs.docker.com/compose/

Expo. (n.d.). *Expo documentation*. Retrieved October 7, 2026, from https://docs.expo.dev/

F5, Inc. (n.d.). *nginx documentation*. Retrieved October 7, 2026, from https://nginx.org/en/docs/

OpenJS Foundation. (n.d.). *Node.js documentation*. Retrieved October 7, 2026, from https://nodejs.org/docs/latest/api/

The PostgreSQL Global Development Group. (n.d.). *PostgreSQL 16 documentation*. Retrieved October 7, 2026, from https://www.postgresql.org/docs/16/